

pifanctl v1 Release Acceptance Field Manual

Status: Trial observations recorded on 2026-10-04 **Release target:** 1.0.0 **Current implementation:** 1.0.0-alpha.1

Decision: Not ready for a stable release. Hardware-specific acceptance remains open.

This manual records the current MicroK8s trial, the bounded temperature-response test, the measured evidence, and the remaining release checks. Cluster-reported temperatures and requested duty are not electrical or RPM measurements.

Values unavailable from Kubernetes are explicitly marked **Not recorded** instead of being guessed.

1. Current trial configuration

Item	Observed value
Kubernetes context	microk8s
Trial namespace	pifanctl-v1-trial
Nodes	raspi-40, raspi-41, raspi-50, raspi-51
Node platforms	ARM64, Kubernetes v1.36.2, Debian GNU/Linux 12, Linux 6.12.93+rpt-rpi-* kernels
Known board labels	raspi-40 and raspi-41: Raspberry Pi 4 Model B; raspi-50: Raspberry Pi 5 Model B; raspi-51: model label not present
Operator / agent / worker image	ghcr.io/jyje/pifanctl-issue:887f6f1-py312
Image digest	sha256:525ef9f01f7bd4d5af5ac4d4014d9f0320187628c41cd2eacd028d5fbb896cf5
Source revision	887f6f1ff05301255e7a5f01e22e117ef8e8e9ef
Runtime chart	pifanctl chart 0.2.0-alpha.1; topology source preserved from 837b3d5
Image build	GitHub Actions run #25 , Python 3.12, passed
Current topology	One shared rack fan cools the four Nodes in CoolingZone r4spi-rack
Actuator	Fan r4spi-rack-fan on raspi-40, RPi.GPIO BCM pin 18, 1000 Hz
Curve	Idle 0%; starts at 30% at 50°C; reaches 100% at 70°C; downward changes are limited to 5 percentage points per refresh
Safety commands	Failsafe duty 100%; exit duty 100%; refresh every 5 seconds; worker heartbeat timeout 120 seconds
Topology hash	eb8b2a42289be4271f52103db3c7980ce5c9a0bb06fe83bc7f17d804b5c3f37c
Readiness	Argo CD Synced/Healthy; operator, four agents, worker, Fan, and CoolingZone reported Ready after rollout
Immediate image rollback	ghcr.io/jyje/pifanctl-issue:6a0f5a5-py312, runtime chart source 837b3d5; the archived v0 configuration remains the full rollback target

At the last recorded sample, 2026-10-04T13:23:40Z, the Fan resource reported a 51.25°C control temperature and a 34.375% requested duty. The measured node temperatures and Fan status vary over time; see the timestamped sample file rather than treating this point as a steady-state result.

2. Filled hardware inventory

The table describes the deployed shared-fan controller. The thermal workload test ran on member raspi-41, which has no local fan in this topology.

Field	Recorded value
Controller Raspberry Pi model	Raspberry Pi 4 Model B, based on the Kubernetes node label; exact PCB revision not recorded
Controller OS, kernel, firmware	Debian GNU/Linux 12; kernel 6.12.93+rpt-rpi-v8; firmware version not recorded
Controller Node and UID	raspi-40; 2b557e96-e346-4e26-b053-04d3cfaa094e
Driver and PWM channel	RPi.GPIO, BCM GPIO 18, 1000 Hz
Fan manufacturer, model, rated voltage/current	Not recorded in Kubernetes; physical fan label or datasheet inspection required
Fan supply, fuse, and current limit	Not recorded; physical wiring and supply inspection required

Field	Recorded value
PWM input type and polarity	Not recorded; verify from the exact fan datasheet and measured waveform
Ground/reference and signal wiring	Not recorded; inspect the physical rack
Independent hardware default and measured signal state	Not recorded; oscilloscope or logic-analyzer measurement required
Tachometer/RPM	No RPM metric or tachometer record was available
Visual rotation after the image update	Awaiting direct post-upgrade confirmation; normal rotation was confirmed before the image update
Test member Node and UID	raspi-41; cabfc25d-6991-427c-a941-3ff323fd6531; Raspberry Pi 4 Model B label

100% is a requested PWM duty, not measured voltage, RPM, or proof that the fan has power. A floating input or GPIO HIGH must not be assumed to mean maximum cooling. No universal pull-up, pull-down, or polarity is prescribed. Verify a hardware safe default and independently powered fan circuit for the actual wiring. Software cannot correct fan-supply loss or mechanical failure.

3. Temperature-response test

Method and guardrails

- The live Fan stayed enabled throughout the CPU-load test. No CR, ConfigMap, Helm value, or temperature threshold was changed.
- Temporary, non-privileged CPU-load Pods were pinned to `raspi-41` and used the deployed candidate image. Each stage had a CPU and memory limit and exited automatically after its fixed duration.
- Prometheus temperatures and Fan status were sampled every 10 to 15 seconds. The load test was set to stop if any member reached 65°C or if temperature/status monitoring failed. Neither stop condition was reached.
- The 65°C abort threshold was a conservative test guardrail, not a pifanctl release setting. Raspberry Pi documents SoC thermal throttling between 80°C and 85°C; this test stayed well below that range. [Raspberry Pi hardware documentation](#)
- The raw records are in [thermal-load-observations.csv](#). They contain Prometheus CPU-thermal readings and contemporaneous Fan resource status. The Prometheus samples were fresh, with observed sample age below 1.1 seconds.
- The CSV has 68 observations from 2026-10-04T12:54:19Z through 2026-10-04T13:23:40Z. SHA-256: 78a3226fbfc82a8cced5c18061e08d256909d11a80333ae9def67f2ba81794f5.

Results

Stage	Applied load	Highest member reading	raspi-41 reading	Fan resource observation	Result
A	1 vCPU, up to 150 seconds	51.8°C	49.173°C maximum	Requested duty 29.375% to 36.3%	Partial; 55°C band not reached
B	2 vCPU, up to 120 seconds	52.35°C	50.147°C maximum	Requested duty reached 38.225%	Partial; brief readings only, no stable hold
C	3 vCPU, up to 120 seconds	55.1°C on raspi-51	49.173°C maximum	At control temperature 55.1°C, status requested 47.85%	Partial; hottest member was not the load target and did not remain at 55.1°C
D	Five-minute no-load observation, begun about seven minutes after the final stress stage	54.55°C maximum during the interval	Recorded separately in the CSV	Control temperature 49.05°C to 54.55°C; requested duty 17.45% to 45.925%	No stable plateau established; this delayed interval does not measure the immediate cooldown transient

The expected steady-state commands from the configured curve are 30% at 50°C, 47.5% at 55°C, 65% at 60°C, 82.5% at 65°C, and 100% at 70°C. The trial reached a short 55.1°C control-temperature snapshot and reported 47.85%, consistent with the configured curve. It did not reach 60°C or higher, and no temperature band was held long enough to claim thermal stabilization.

The test is not an isolated causal measurement. The hottest readings came from `raspi-51`, not the stress target `raspi-41`, and the cluster hosts other services. The stress target peaked at 50.147°C, while the hottest rack member peaked at 55.1°C. A later `kubect l top` nodes sample showed about 19% CPU on both `raspi-40` and `raspi-41`, and about 6% on

raspi-50 and raspi-51, after the load had ended. The data demonstrate changing cluster telemetry and corresponding Fan status, but they do not prove that the injected load alone caused the raspi-51 temperature peak.

Measured MicroK8s temperature and fan-command response

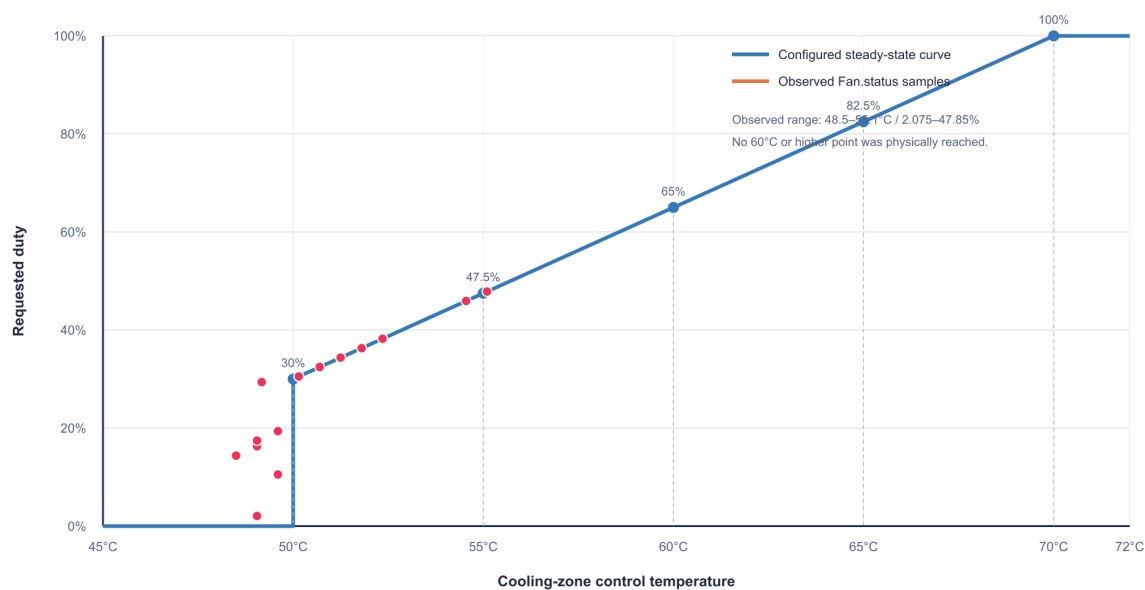
Node CPU thermal sensor samples; Fan resource values are requested commands, not measured RPM or PWM voltage.



This figure plots the node selected for load, the concurrently hottest node, the cluster maximum, the Fan status control temperature, and the requested duty. Gaps between test stages remain gaps in time. Fan status is not measured PWM voltage or RPM.

Configured control curve and observed duty commands

Hypothesis: linear increase from 30% at 50°C to 100% at 70°C; actual status includes the 5% per-refresh down-step.



The solid line is the configured rising curve. Markers are observed Fan status snapshots. The downward ramp can lag the curve because the configured controller limits each decrease to 5 percentage points per refresh. Target points at 60°C and

above are curve calculations, not live measurements.

Stability criteria and interpretation

For a future acceptance run, call a temperature band stable only after the hottest member stays within a 1°C range for at least 120 seconds under a declared, repeatable workload, telemetry remains fresh, and requested duty has no unexplained increase. Record the fan RPM or direct visual rotation during the entire run. This trial did not satisfy that criterion. The member temperature readings varied independently, and the hottest member was outside the node receiving the test load. The recorded five-minute no-load interval began about seven minutes after the last CPU-load sample, so it is not evidence for the immediate cooling slope or time-to-stability.

The live test ended below the 65°C abort guardrail, all temporary load Pods were removed, and no trial configuration was changed. The physical-fan-stop and recovery test has **not** been run. Its safe abort threshold is awaiting confirmation; the hardware default, RPM, and post-upgrade physical rotation also remain unverified.

4. Other scenario records

Scenario	Current evidence	Status
Normal shared-rack regulation	Fan and CoolingZone Ready; Fan status follows the hottest available member samples and reports requested duty	Observed; not a stable thermal acceptance run
Local node CPU load	Three capped CPU-load stages on <code>raspi-41</code> ; see Section 3 and CSV	Partial; did not hold a target temperature
Temperature missing or stale	The worker is configured for fail-safe 100%; no live telemetry fault was injected	Not run live; automated mock/API scenarios cover this behavior
Operator heartbeat expires	Worker plan uses a 120-second heartbeat timeout and fail-safe 100%; no live heartbeat fault was injected	Not run live
Physical shared fan stopped, then restored	No stop command or fan-power interruption was applied; actual fan circuit default is not yet documented	Not run; safety guardrail confirmation pending
Controller process exit, reboot, or power loss	No live fault was injected; process and electrical defaults need measurement	Not run
Pi 5 PWM hardware	<code>raspi-50</code> is labelled Pi 5 Model B, but the trial fan actuator uses RPi.GPIO on Pi 4 <code>raspi-40</code>	Not run on Pi 5 hardware

The examples below describe expected control behavior, not measured thermal traces:

- **Normal load:** the shared fan follows the hottest member. A hotter member raises the shared fan request while cooler members remain in the same CoolingZone.
- **One hot member:** the selected zone's fan should rise along the configured curve; unrelated zones should not change.
- **Missing temperature or expired heartbeat:** the affected worker should request 100% and report an unhealthy reason. No thermal rise is predicted here because the live faults were not injected.
- **Fan stops or loses supply:** a software duty command cannot guarantee cooling. The measured hardware default and restoration behavior must be recorded before this scenario can pass.

5. Remaining release procedures

Pi 4 GPIO PWM waveform and physical behavior

Record the actual board revision, OS/firmware, fan part number, supply/current protection, wiring, polarity, voltage levels, and instrument setup. With a person at the hardware, capture startup, requested duty changes, fan start-from-rest, normal stop, and process termination. Repeat at least three cold starts and three normal stops. **Pass only when measured waveform and physical fan behavior agree with the fan datasheet at all tested points.**

Pi 5 kernel PWM

Verify the actual overlay, PWM chip/channel, pin mux, polarity, and frequency on the physical Pi 5. The sample `chip: 0, channel: 2` in `design/v1/examples/sysfs.yaml` is illustrative. Check single-writer ownership and repeat startup, duty, reboot, and shutdown measurements. Do not infer a Pi model or pin map from a Node name.

Process, reboot, power, and network faults

In an isolated maintenance window, test graceful stop and SIGKILL separately, then orderly reboot, abrupt controller power loss, fan-supply interruption, worker/operator partition, and Prometheus loss. Record physical rotation, scope/tachometer data, failsafe time, /readyz, status reason, and recovery. Never count Python cleanup as protection against SIGKILL.

Migration, rollback, and fleet load

Capture the archived v0 configuration and checksum. Inventory every writer of each PWM channel. Compare a read-only v1 plan to the physical rack, keep cooling at the safe state during handover, stop the old writer and verify lock release before starting v1, then exercise rollback. Measure CPU/memory, API requests, reconciliation duration, status writes, Prometheus latency, scrape gaps, and heartbeat age at a declared fleet size.

6. Evidence and release decision

Record	Candidate and method	Result	Evidence
Deployment	887f6f1-py312, digest recorded in Section 1	Argo CD Synced/Healthy; Fan and CoolingZone Ready	Workflow run #25
Load stage A	raspi-41, 1 vCPU, maximum 150 seconds	Partial; maximum zone sample 51.8°C	Raw samples
Load stage B	raspi-41, 2 vCPU, maximum 120 seconds	Partial; maximum zone sample 52.35°C	Raw samples
Load stage C	raspi-41, 3 vCPU, maximum 120 seconds	Partial; 55.1°C transient on raspi-51; no steady hold	Raw samples
Cooldown	Five-minute no-load observation, started about seven minutes after the final load sample	Not stable; immediate cooldown transient was not captured	Raw samples
Electrical waveform, RPM, and physical fan-off recovery	No instruments or completed post-upgrade observation recorded	Not run	Physical acceptance evidence required
Pi 5 hardware, fault injection, rollback, fleet scale	No live trial records	Not run	Procedures in Sections 5 and 3

Do not label 1.0.0 stable until every applicable release gate has evidence for the exact release candidate. Hardware or topologies not tested must be listed as unsupported. Preserve the v0 rollback archive until the release decision is recorded.

References

- [Runtime manual](#)
- [v1 architecture and acceptance design](#)
- [Implementation and release checklist](#)
- [Raspberry Pi frequency and thermal management](#)
- [RPi.GPIO project](#)
- [Linux kernel PWM interface](#)